



학습 목표

1. C++의 표준 입출력 방식인 스트림 입출력에 대해 이해한다.
2. ostream의 멤버 함수를 이용하여 문자를 출력하는 방법을 안다.
3. istream의 멤버 함수를 이용하여 문자, 문자열, 한 줄 입력 받는 방법을 안다.
4. 포맷 입출력 방법을 알고 프로그램을 작성할 수 있다.
5. 조작자를 이해하고, 조작자의 실행 과정을 안다.
6. 새로운 조작자를 작성할 수 있다.
7. 삽입연산자(<<)와 추출연산자(>>)의 실행 과정을 안다.
8. 새로운 삽입연산자와 추출연산자를 작성할 수 있다.

스트림

3

□ 스트림(stream)

- ▣ 데이터의 흐름, 혹은 데이터를 전송하는 소프트웨어 모듈

- 흐르는 시내와 유사한 개념

- ▣ 스트림의 양 끝에는 프로그램과 장치 연결

- 보낸 순서대로 데이터 전달
- 입출력 기본 단위 : 바이트

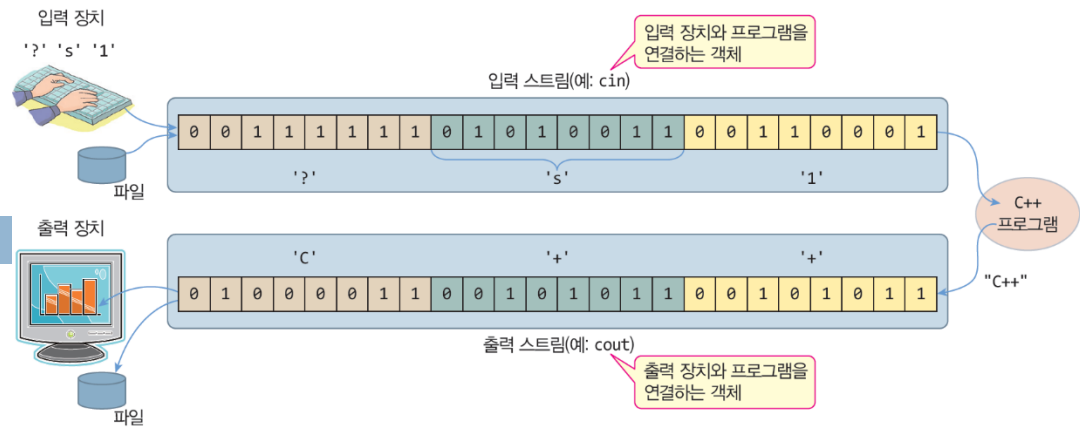
□ C++ 스트림 종류

- ▣ 입력 스트림

- 입력 장치, 네트워크, 파일로부터 데이터를 프로그램으로 전달하는 스트림

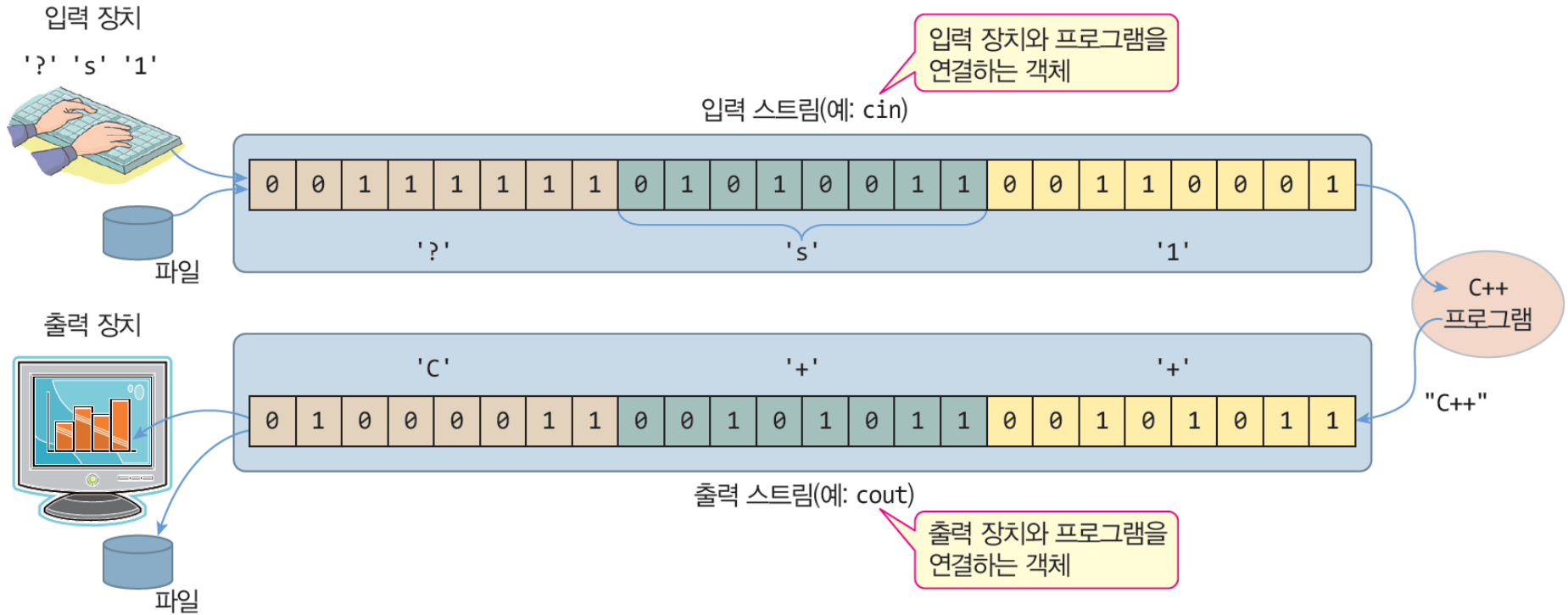
- ▣ 출력 스트림

- 프로그램에서 출력되는 데이터를 출력 장치, 네트워크, 파일로 전달하는 스트림



C++ 입출력 스트림

4



C++ 입출력 스트림 버퍼

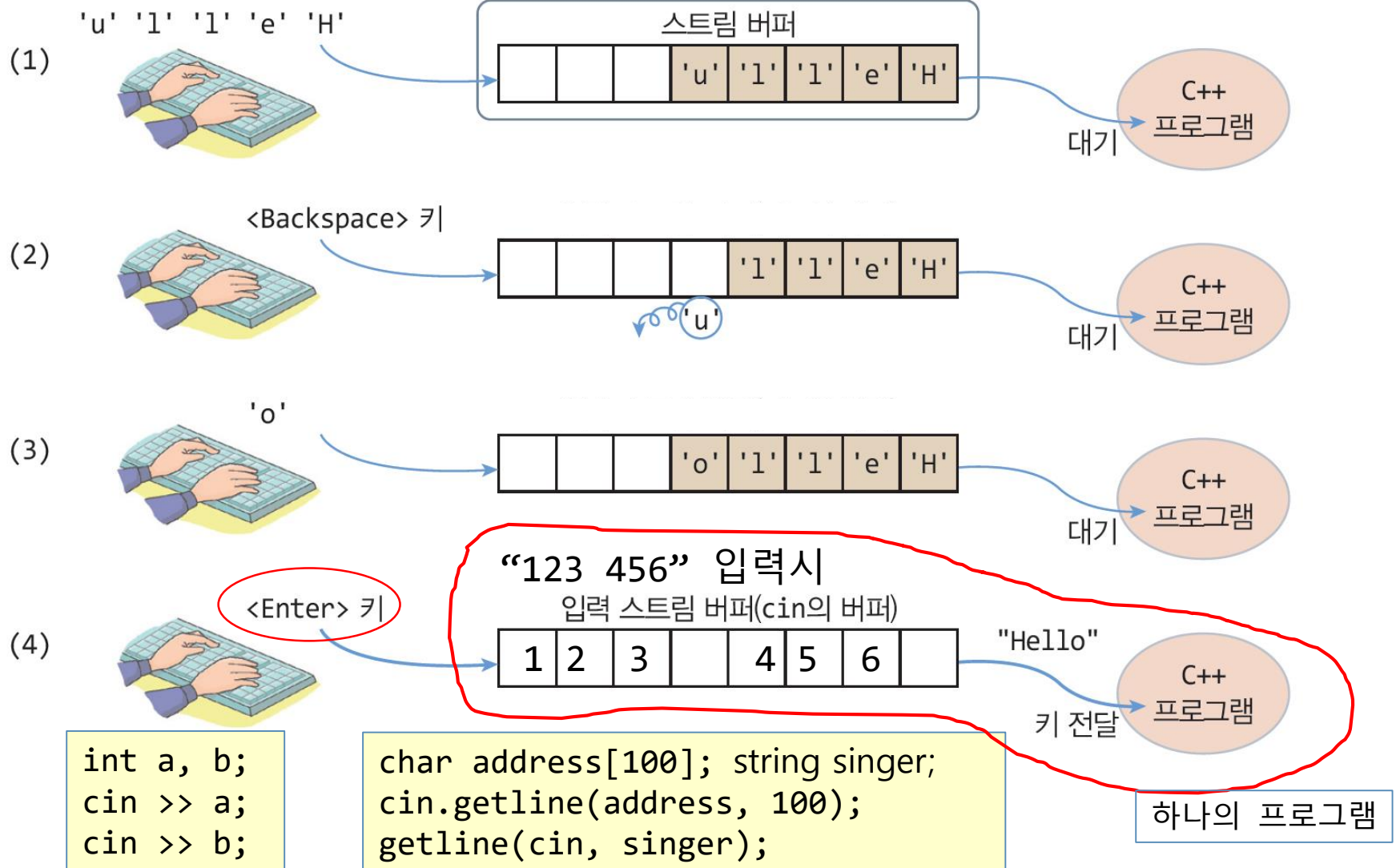
5

- C++ 입출력 스트림은 버퍼를 가짐 (char 배열)
- 키 입력 스트림의 버퍼
 - ▣ 목적
 - 입력장치로부터 입력된 데이터를 프로그램으로 전달하기 전에 일시 저장
 - 키 입력 도중 수정 가능
 - <Backspace> 키가 입력되면 이전에 입력된 키를 버퍼에서 지움
 - ▣ 프로그램은 사용자의 키 입력이 끝난 시점에서 읽음
 - <Enter> 키 : 키 입력의 끝을 의미
 - <Enter> 키가 입력된 시점부터 키 입력 버퍼에서 프로그램이 읽기 시작
- 스크린 출력 스트림 버퍼
 - ▣ 목적
 - 프로그램에서 출력된 데이터를 출력 장치로 보내기 전에 일시 저장
 - 출력 장치를 반복적으로 사용하는 비효율성 개선
 - ▣ 버퍼가 꽉 차거나 강제 출력 명령 시에 출력 장치에 출력

키 입력 스트림과 버퍼의 역할

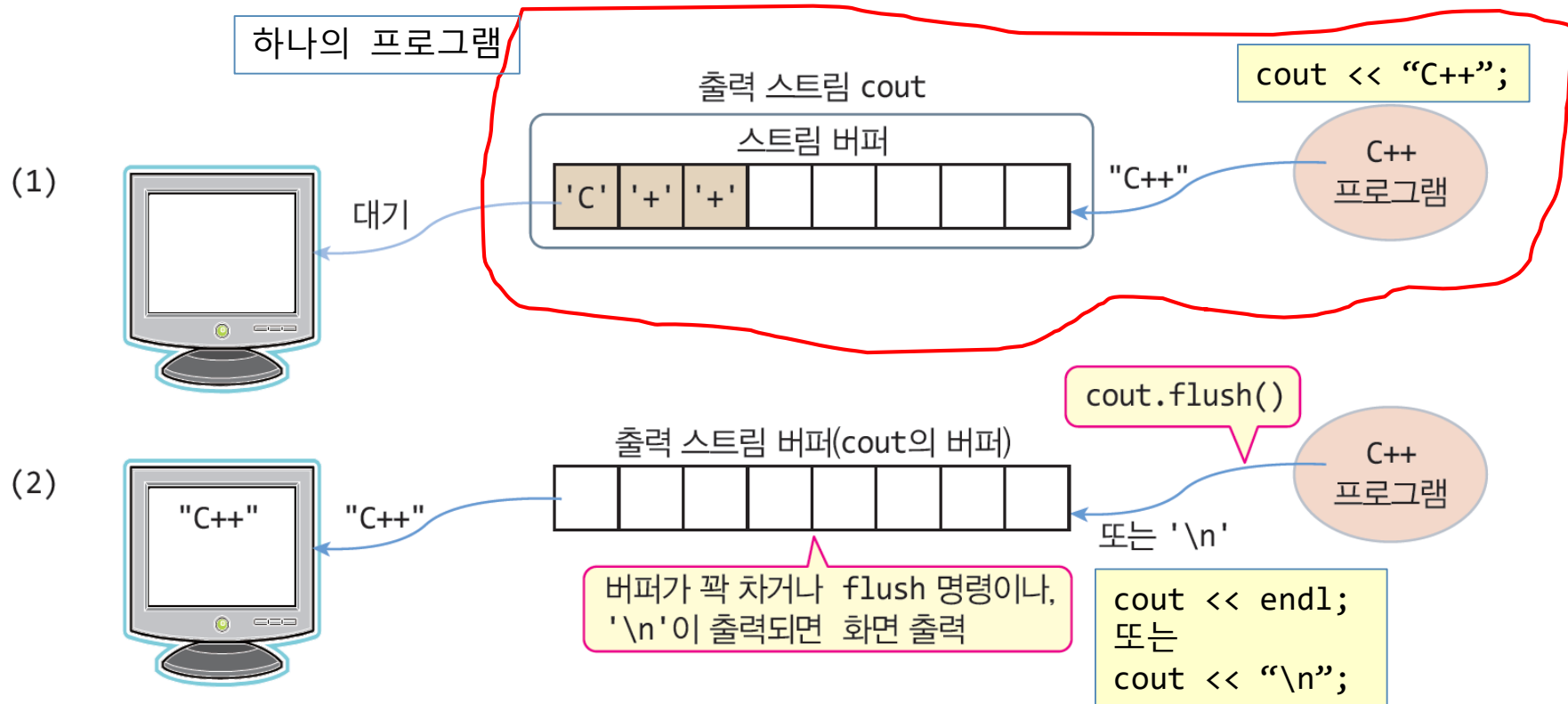
6

키보드 디바이스



스크린 출력 스트림과 버퍼의 역할

7



C++ 표준은 스트림 입출력만 지원

8

□ 입출력 방식 2가지

▣ 스트림 입출력 방식(stream I/O)

- 스트림 버퍼를 이용한 입출력 방식
- 입력된 키는 버퍼에 저장
 - <Enter>키가 입력되면 프로그램의 버퍼로 한 줄 전체가 읽혀 가는 방식
- 출력되는 데이터는 일차적으로 스트림 버퍼에 저장
 - 버퍼가 꽉 차거나, '\n'을 만나거나, 강제 출력 명령의 경우에만 버퍼에서 출력 장치에 출력

▣ 저 수준 입출력 방식(raw level console I/O)

- 키가 입력되는 즉시 프로그램에게 키 값 전달
 - <Backspace>키 그 자체도 프로그램에게 바로 전달
 - 게임 등 키 입력이 즉각적으로 필요한 곳에 사용
- 프로그램이 출력하는 즉시 출력 장치에 출력
- 컴파일러마다 다른 라이브러리나 API 지원
 - C++ 프로그램의 호환성 낮음

□ C++ 표준은 스트림 방식만 지원

- 스트림 입출력은 모든 표준 C++ 컴파일러에 의해 컴파일됨
- 높은 호환성

2003년 이전의 C++ 입출력 라이브러리의 약점

9

- 대표적인 구 표준(C++03) 입출력 라이브러리 클래스
 - ▣ ios, istream, ostream, iostream, ifstream, ofstream, fstream
- 문자를 한 바이트의 char로 처리
 - ▣ cin >>로 문자를 읽을 때, 한글 문자 읽을 수 없음
 - 영어나 기호 : 1 바이트
 - 한글 문자 : 2 바이트

```
char ch;  
cin >> ch;      // 키보드로부터 문자 입력. 한글 문자 읽을 수 없음  
char name[20]; // 문자열 또는 string name;으로 읽음  
cin >> name;    // 키보드로부터 한글 문자 읽을 수 있음
```

- 2020년 지금도 마찬가지로 cin으로 한글을 문자 단위로 읽을 수 없음

새 표준 C++ 입출력 라이브러리

10

- 다양한 크기의 다국어 문자를 수용하기 위해, 입출력 라이브러리를 템플릿으로 작성

스트림 입출력
기본 템플릿 클래스

basic_ios

입출력 스트림
템플릿 클래스

basic_istream

basic_ostream

basic_iostream

파일 입출력
템플릿 클래스

basic_ifstream

basic_ofstream

basic_fstream

새 표준 입출력 라이브러리 - 템플릿으로 작성

ios

istream

ostream

iostream

ifstream

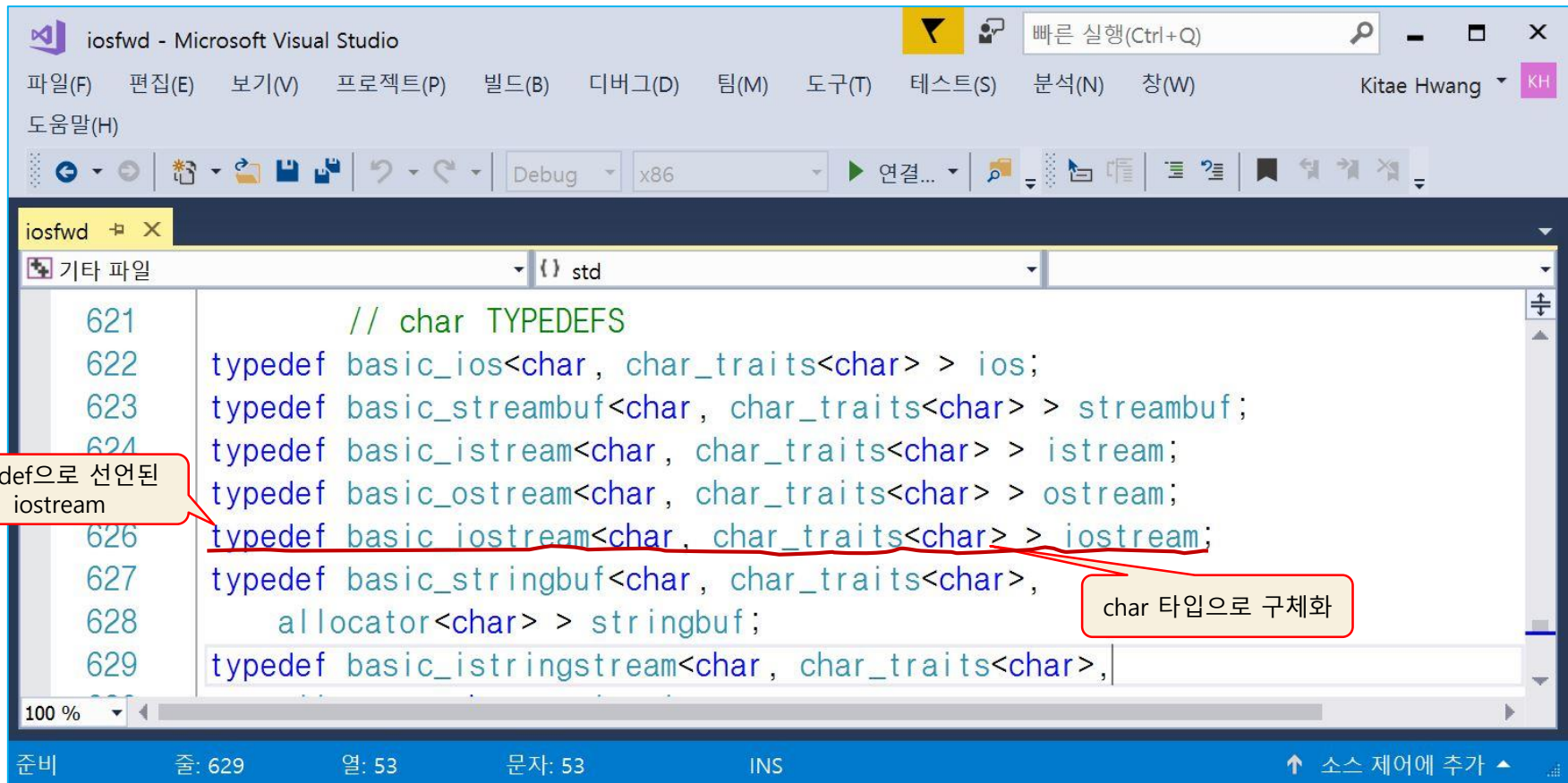
ofstream

fstream

템플릿에 char 타입으로 구체화한 클래스
- 구 표준의 이름 그대로 사용할 수 있음

typedef로 선언된 ios, istream, ostream, iostream 클래스

11



입출력 클래스 소개

12

클래스	설명
ios	모든 입출력 스트림 클래스들의 기본(Base) 클래스. 스트림 입출력에 필요한 공통 함수와 상수, 멤버 변수 선언
istream, ostream, iostream	istream은 문자 단위 입력 스트림. ostream은 문자 단위 출력 스트림. iostream은 문자 단위로 입출력을 동시에 할 수 있는 스트림 클래스
ifstream, ofstream, fstream	파일에서 읽고 쓰는 기능을 가진 파일 입출력 스트림 클래스. 파일에서 읽을 때는 ifstream 클래스를, 파일에 쓸 때는 ofstream 클래스를, 읽고 쓰기를 동시에 할 때 fstream 클래스 이용

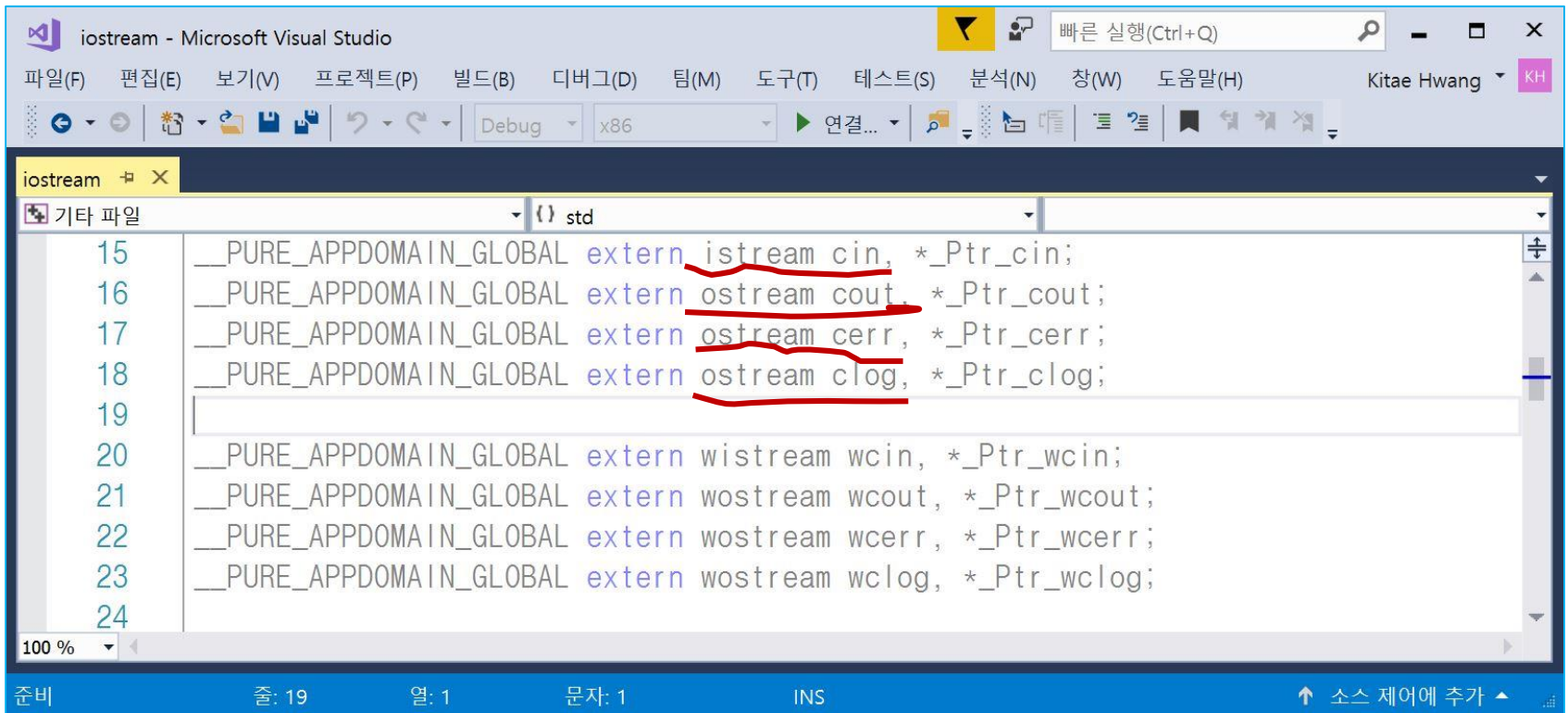
표준 입출력 스트림 객체

13

- C++ 프로그램이 실행될 때 자동으로 생겨나는 스트림
 - ▣ cin
 - istream타입의 스트림 객체로서 키보드 장치와 연결
 - ▣ cout
 - ostream타입의 스트림 객체로서 스크린 장치와 연결
 - ▣ cerr
 - ostream타입의 스트림 객체로서 스크린 장치와 연결
 - 오류 메시지를 출력할 목적
 - 스트림 내부 버퍼 거치지 않고 출력
 - ▣ clog
 - ostream타입의 스트림 객체로서 스크린 장치와 연결
 - 오류 메시지 또는 진행 경과 메시지를 출력할 목적
 - 스트림 내부에 버퍼 거쳐 출력

<iostream>에 선언된 스트림 객체들

14



```
iostream - Microsoft Visual Studio
빠른 실행(Ctrl+Q)

파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 팀(M) 도구(T) 테스트(S) 분석(N) 창(W) 도움말(H) Kitae Hwang KH

Debug x86 연결...

iostream
기타 파일 {} std
15 __PURE_APPDOMAIN_GLOBAL extern istream cin, *_Ptr_cin;
16 __PURE_APPDOMAIN_GLOBAL extern ostream cout, *_Ptr_cout;
17 __PURE_APPDOMAIN_GLOBAL extern ostream cerr, *_Ptr_cerr;
18 __PURE_APPDOMAIN_GLOBAL extern ostream clog, *_Ptr_clog;
19
20 __PURE_APPDOMAIN_GLOBAL extern wistream wcin, *_Ptr_wcin;
21 __PURE_APPDOMAIN_GLOBAL extern wostream wcout, *_Ptr_wcout;
22 __PURE_APPDOMAIN_GLOBAL extern wostream wcerr, *_Ptr_wcerr;
23 __PURE_APPDOMAIN_GLOBAL extern wostream wclog, *_Ptr_wclog;
24
100 %
준비 줄: 19 열: 1 문자: 1 INS ↑ 소스 제어에 추가 ▲
```

ostream 멤버 함수

15

```
// "<<" 연산자를 이용한 출력
int i = 3; double d = 2.3; char c = 'c'; string s("C++");
char ca1[] = {'C', '+', '+' };   char ca2[] = "C++";
cout << i << " " << d << " " << c << " " << s << ca1 << " " << ca2;
// 출력: 3 2.3 c C++ C++ C++
```

ostream& put(char ch)

*ch*의 문자를 스트림에 출력

ostream& write(char str, int n)*

str 배열에 있는 *n*개의 문자를 스트림에 출력

ostream& flush()

현재 스트림 버퍼에 있는 내용 강제 출력

```
// 멤버 함수를 이용한 출력
cout.put('H'); cout.put(33); // ASCII 코드 33은 '!' 문자임
cout.put('\n');
char str[]="I love programming";
cout.write(str, 6); // str 배열의 6 개의 문자 "I love"를 스트림에 출력
cout.write(str, strlen(str)); // str 배열 전체를 스트림에 출력
cout.flush(); // cout 스트림 버퍼에 저장된 내용을 강제로 스크린에 출력

cout.put('H').put('\n').write(str, 6).write(str, strlen(str)).flush();
```

예제 11-1 ostream 멤버 함수를 이용한 문자 출력

16

ostream의 put(), write() 멤버 함수를 이용하여 문자를 화면에 출력하는 사례를 보여준다.

```
int main() {
    // "Hi!"를 출력하고 다음 줄로 넘어간다.
    cout.put('H');
    cout.put('i');
    cout.put(33);
    cout.put('\n');

    // "C++ "을 출력한다.
    cout.put('C').put('+').put('+').put(' ');

    char str[]="I love programming";
    cout.write(str, 6); // str 배열의 6 개의 문자 "I love"를 스트림에 출력
    cout.write(str, strlen(str)); // str 배열 전체를 스트림에 출력
    for (int i = 0; str[i] != '\0'; ++i)
        cout.put(str[i]);
    cout.flush();
}
```

ASCII 코드 33은 '!' 문자임

put() 메소드를 연결하여 사용할 수 있다.

```
int i;
while (str[i] != '\0')
    cout.put(str[i++]);
```

```
Hi!
C++ I love
I love programming
I love programming
```


istream 멤버 함수 – 문자 입력, get() 함수

17

```
int get()
```

입력 스트림에서 문자를 읽어 리턴. 오류나 EOF를 만나면 -1(EOF) 리턴

```
istream& get(char& ch)
```

cin의 참조가 리턴됨

입력 스트림에서 문자를 읽어 ch에 저장. 현재 입력 스트림 객체(*this)의 참조 리턴. 오류나 EOF를 만나면, 스트림 내부의 오류 플래그(failbit) 세팅(12.7절 참조)

```
char ch;  
// 셋 모두 동일  
cin >> ch;  
ch = cin.get();  
cin.get(ch);
```

- int get()을 이용하여 한 라인의 문자들을 읽는 코드

```
int ch;  
while((ch = cin.get()) != EOF) { // EOF 는 -1  
    cout.put(ch); // 읽은 문자 출력  
    if(ch == '\n')  
        break; // <Enter> 키가 입력되면 읽기 중단  
}
```

입력 스트림의 끝인지 비교
Ctrl+Z 또는 Ctrl+D를 누를 때

리턴 값이 cin 참조자이고, cin
이 if (cin) 또는 while (cin) 처
럼 조건문에 사용될 때,
(EOF나 오류)가 아니면 true

- istream& get(char& ch)을 이용하여 한 라인의 문자들을 읽는 코드

```
char ch;  
while(true) {  
    cin.get(ch); // 입력된 키를 ch에 저장하여 리턴  
    if(cin.eof()) break; // EOF를 만나면 읽기 종료  
    cout.put(ch); // ch의 문자 출력  
    if(ch == '\n')  
        break; // <Enter> 키가 입력되면 읽기 중단  
}
```

입력 스트림의
끝인지 비교

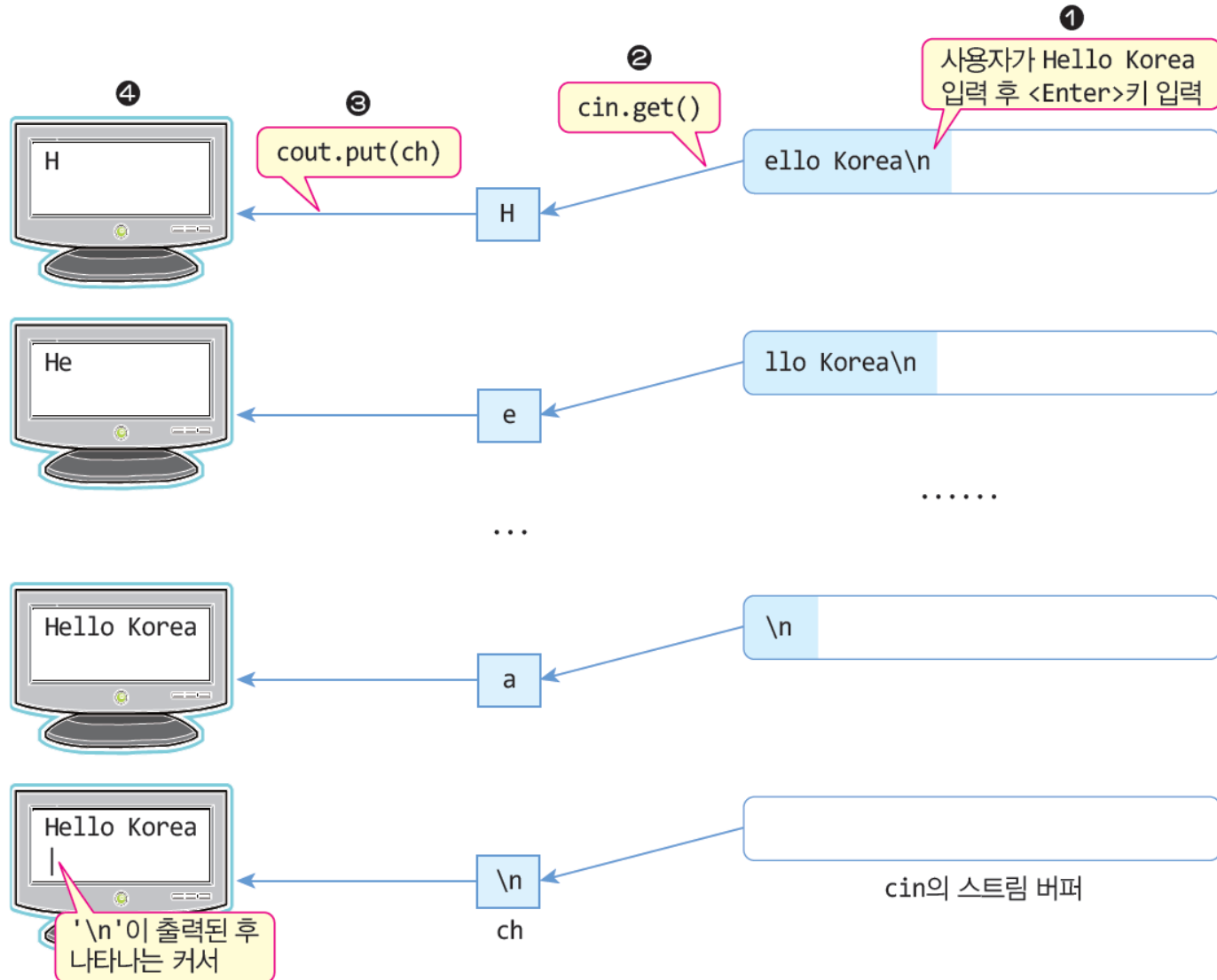
if (!cin.get(ch))
break; // 동일

cin이 if (!cin) 또는 while (!cin) 처럼 조건문에 사용될 때, EOF 또는 오류이면 true

```
char ch;  
while(cin.get(ch)) {  
    cout.put(ch);  
    if(ch == '\n')  
        break;  
}  
while ((cin >> ch))  
if (cin)
```

ch = cin.get()의 반복 실행 사례

18



예제 11-2 get()과 get(char&)을 이용한 한 줄의 문자 읽기

```
#include <iostream>
using namespace std;

void get1() {
    cout << "cin.get()로 <Enter> 키까지 입력 받고 출력합니다>>";
    int ch; // EOF와의 비교를 위해 int 타입으로 선언
    while((ch = cin.get()) != EOF) { // 문자 읽기. EOF 는 -1
        cout.put(ch); // 읽은 문자 출력
        if(ch == '\n')
            break; // <Enter> 키가 입력되면 읽기 중단
    }
}

void get2() {
    cout << "cin.get(char&)로 <Enter> 키까지 입력 받고 출력합니다>>";
    char ch;
    while(true) { // while(cin.get(ch)) 또는 while (cin >> ch) {
        cin.get(ch); // 문자 읽기;
        if(cin.eof()) break; // EOF를 만나면 읽기 종료
        cout.put(ch); // ch의 문자 출력
        if(ch == '\n')
            break; // <Enter> 키가 입력되면 읽기 중단
    }
}

int main() {
    get1(); // cin.get()을 이용하는 사례
    get2(); // cin.get(char&)을 이용하는 사례
}
```

```
cin.get()로 <Enter> 키까지 입력 받고 출력합니다>>Do you love C++?
Do you love C++?
cin.get(char&)로 <Enter> 키까지 입력 받고 출력합니다>>Yes, I do.
Yes, I do.
```

문자열 입력

20

```
istream& get(char* s, int n)
```

입력 스트림으로부터 $n-1$ 개의 문자를 읽어 배열 s 에 저장하고 마지막에 `'\0'` 문자 삽입. 입력 도
중 `'\n'`을 만나면 `'\0'`을 삽입하고 리턴

```
char str[256];  
cin.get(str, 256); // 최대 255개의 문자를 읽고 끝에 '\0'를 붙여 str 배열에 저장  
cout << str << endl; // str을 화면에 출력
```

문자열 입력 시 주의사항

21

- 입력 도중 <Enter>키('\n')을 만날 때 읽기를 중단하고 리턴
 - <Enter> 키('\n')를 스트림 버퍼에 남김
 - 다시 `get(s, n)`으로 문자열 읽기를 시도하면 입력 스트림에 남은 '\n'키를 읽게 되어 무한 루프에 빠짐
 - `cin.get()`이나 `cin.ignore(1);`를 통해 문자 1개('\n')를 스트림에서 읽어 버려야 함.

```
char line[80];
while (true) {
    cin.get(line, 80);
    if ("exit" 입력이면)
        break;
    cout << line << endl;
    cin.ignore(1);
}
```

`cin.get(line, 80);`

C++ programming language.

C++ programming language.\n

<Enter>키를 입력하면
읽기 시작됨

.....

출력할 때는 다시 'Wn'을 출력해야 함

.....

\n

cin의 스트림 버퍼

C++ programming language.\0

char line[80]

\n이 남아 있음에 주의

예제 11-3 get(char*, int)을 이용한 문자열 입력

22

"exit"이 입력되면 프로그램을 종료하도록 작성하라.

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char cmd[80];
    cout << "cin.get(char*, int)로 문자열을 읽습니다." << endl;
    while(true) {
        cout << "종료하려면 exit을 입력하세요 >> ";
        cin.get(cmd, 80); // 79개까지의 영어 문자 읽음.
        // cin.get(cmd, 80, '\n');와 동일함
        if(strcmp(cmd, "exit") == 0) { // "exit" 입력이면
            cout << "프로그램을 종료합니다....";
            return 0;
        }
        else
            cin.ignore(1); // 버퍼에 남아 있는 <Enter> 키 ('\n') 제거
    }
}
```

'\n'은 입력 스트림 버퍼에 남겨둠

39개까지의 한글 문자 읽을 수 있음

입력 버퍼에 남아 있는 '\n' 제거

이 부분을 제거하면 무한 루프에 빠짐

```
cin.get(char*, int)로 문자열을 읽습니다.
종료하려면 exit을 입력하세요 >> exi
종료하려면 exit을 입력하세요 >> exiT
종료하려면 exit을 입력하세요 >> exito
종료하려면 exit을 입력하세요 >> exit
프로그램을 종료합니다....
```

한 줄 읽기

23

```
char line[80];  
cin.getline(line, 80);  
// 위와 동일  
string ln;  
getline(cin, ln);
```

```
istream& get(char* s, int n, char delim='\n')
```

입력 스트림으로부터 최대 $n-1$ 개의 문자를 읽어 배열 s 에 저장하고 마지막에 $\backslash 0$ 문자 삽입. 입력
도중 $delim$ 에 지정된 구분 문자를 만나면 지금까지 읽은 문자를 배열 s 에 저장하고 리턴

```
istream& getline(char* s, int n, char delim='\n')
```

get()과 동일. 하지만 $delim$ 에 지정된 구분 문자를 스트림에서 제거

```
char line[80];  
while (true) {  
    cin.getline(line, 80);  
    if ("exit" 입력이면)  
        break;  
    cout << line << endl;  
    cin.ignore(1);  
}
```

'\n'은 line[]에 삽입하지 않고, 스트림 버퍼에서 제거

cin.getline(line, 80);

출력할 때는 다시 '\n'을 출력해야 함

C++ programming language.

C++ programming language.\n

C

<Enter>키를 입력하면
읽기 시작됨

'\n'은 스트림 버퍼에서
제거 되었기 때문에, 이
문장은 더 이상 필요 없음

C++ programming language.\0

char line[80]

cin의 스트림 버퍼

\n이 없음에 주목

예제 11-4 getline()으로 한 줄 단위로 문장 읽기

24

istream의 getline()을 이용하여 빈 칸을 포함하는 한 줄을 읽고 다시 그대로 출력하는 프로그램을 작성하라.

```
#include <iostream>
using namespace std;

int main() {
    char line[80];
    cout << "cin.getline() 함수로 라인을 읽습니다." << endl;
    cout << "exit를 입력하면 루프가 끝납니다." << endl;

    int no = 1; // 라인 번호
    while(true) {
        cout << "라인 " << no << " >> ";
        cin.getline(line, 80); // 79개까지의 문자 읽음
        if(strcmp(line, "exit") == 0)
            break;
        cout << "echo --> ";
        cout << line << endl; // 읽은 라인을 화면에 출력
        no++; // 라인 번호 증가
    }
}
```

```
cin.getline() 함수로 라인을 읽습니다.
exit를 입력하면 루프가 끝납니다.
라인 1 >> It's now or never.
echo --> It's now or never.
라인 2 >> Come hold me tight.
echo --> Come hold me tight.
라인 3 >> Kiss me my darling, be mine tonight.
echo --> Kiss me my darling, be mine tonight.
라인 4 >> 엘비스 프레슬리 노래입니다.
echo --> 엘비스 프레슬리 노래입니다.
라인 5 >> exit
```

'\n'은 line[]에 삽입하지 않고, 스트림 버퍼에서 제거

입력 문자 건너 띄기와 문자 개수 알아내기

25

```
istream& ignore(int n=1, int delim=EOF)
```

입력 스트림에서 n개 문자 제거. 도중에 delim 문자를 만나면 delim 문자를 제거하고 리턴

```
int gcount()
```

최근에 입력 스트림에서 읽은 바이트 수(문자의 개수) 리턴. <Enter> 키도 개수에 포함

▣ 입력 스트림에서 문자 건너뛰기

```
cin.ignore();           // 입력 스트림에 입력된 문자 중 1개 제거
cin.ignore(10);         // 입력 스트림에 입력된 문자 중 10개 제거

cin.ignore(10, ';');    // 입력 스트림에서 10개의 문자 제거.
                        // 제거 도중 ';'을 만나면 ';'을 제거하고 리턴
```

▣ 최근에 읽은 문자 개수 리턴

```
char line[80];
cin.getline(line, 80);
int n = cin.gcount(); // 최근의 실행한 getline() 함수에서 읽은 문자의 개수 리턴
// 또는 int n = strlen(line);
```

포맷 입출력

26

- C++에서도 입출력 시 포맷 지정 가능
 - ▣ C 언어의 printf()와 유사
- 포맷 입출력 방법 3 가지
 - ▣ 포맷 플래그
 - ▣ 포맷 함수
 - ▣ 조작자

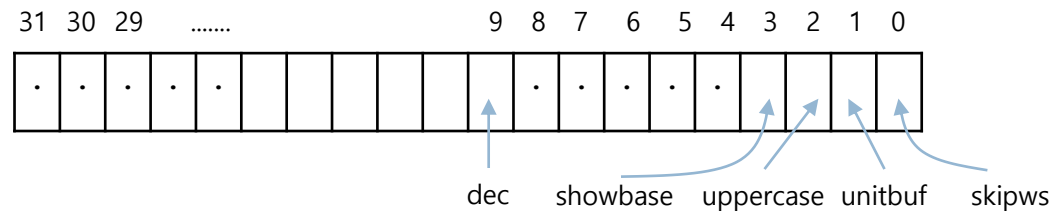
포맷 플래그

27

□ 포맷 플래그

- ▣ 입출력 스트림에서 입출력 형식을 지정하기 위한 플래그

포맷 플래그 값을 가진 ios 클래스의 멤버 변수



ios 클래스에 정의된 포맷 플래그

28

플래그	값	의미
<code>ios::skipws</code>	<code>0x0001</code>	입력시 공백 문자(스페이스, 탭, 개행문자)를 무시
<code>ios::unitbuf</code>	<code>0x0002</code>	출력 스트림에 들어오는 데이터를 버퍼링하지 않고 바로 출력
<code>ios::uppercase</code>	<code>0x0004</code>	16진수의 A~F, 지수 표현의 E를 대문자로 출력
<code>ios::showbase</code>	<code>0x0008</code>	16진수이면 0x를, 8진수이면 0을 숫자 앞에 붙여 출력
<code>ios::showpoint</code>	<code>0x0010</code>	실수 값에 대해, 정수 부분과 더불어 소수점 이하의 끝자리들을 0으로 출력
<code>ios::showpos</code>	<code>0x0020</code>	양수에 대해 + 기호 출력
<code>ios::left</code>	<code>0x0040</code>	필드를 왼쪽 맞춤(left-align) 형식으로 출력
<code>ios::right</code>	<code>0x0080</code>	필드를 오른쪽 맞춤(right-align) 형식으로 출력
<code>ios::internal</code>	<code>0x0100</code>	부호는 왼쪽 맞춤으로 숫자는 오른쪽 맞춤으로 출력
<code>ios::dec</code>	<code>0x0200</code>	10진수로 출력. 디폴트 설정
<code>ios::oct</code>	<code>0x0400</code>	8진수로 출력
<code>ios::hex</code>	<code>0x0800</code>	16진수로 출력
<code>ios::scientific</code>	<code>0x1000</code>	실수에 대해 과학 산술용 규칙에 따라 출력
<code>ios::fixed</code>	<code>0x2000</code>	실수에 대해 소수점 형태로 출력
<code>ios::boolalpha</code>	<code>0x4000</code>	설정되면, 논리값 true를 "true"로, false를 "false"로 출력하고, 설정되지 않으면, 정수 1과 0으로 출력

포맷 플래그를 세팅하는 포맷 멤버 함수

29

Long setf(Long flags)

flags를 스트림의 포맷 플래그로 설정하고 이전 플래그를 리턴한다.

Long unsetf(Long flags)

flags에 설정된 비트 값에 따라 스트림의 포맷 플래그를 해제하고 이전 플래그를 리턴한다.

```
cout.unsetf(ios::dec);    // 10진수 해제
cout.setf(ios::hex);     // 16진수로 설정
cout << 30 << endl;      // 1e가 출력됨
```

```
cout.setf(ios::dec | ios::showpoint); // 10진수 표현과 동시에 실수에
                                         // 소숫점이하 나머지는 0으로 출력
cout << 23.5 << endl;           // 23.5000으로 출력 (꼭 6자리 기본)
```

예제 11-5 setf(), unsetf()를 사용한 포맷 출력

30

```
#include <iostream>
using namespace std;

int main() {
    cout << 30 << endl;           // 10진수로 출력

    cout.unsetf(ios::dec);         // 10진수 해제
    cout.setf(ios::hex);           // 16진수로 설정

    cout << 30 << endl;           // 16진수로 출력

    cout.setf(ios::showbase);      // 16진수로 설정

    cout << 30 << endl;           // 16진수의 A~F는 대문자로 출력

    cout.setf(ios::uppercase);     // 16진수의 A~F는 대문자로 출력

    cout << 30 << endl;           // 16진수로 출력

    cout.unsetf(ios::hex);         // 16진수 해제
    cout.setf(ios::dec | ios::showpoint); // 10진수 표현과 동시에
                                     // 소숫점 이하 나머지는 0으로 출력

    cout << 23.5 << endl;         // 10진수로 출력

    cout.setf(ios::scientific);    // 실수를 과학산술용 표현으로 출력

    cout << 23.5 << endl;         // 2.350000E+001로 출력

    cout.setf(ios::showpos);       // 양수인 경우 + 부호도 함께 출력

    cout << 23.5;                 // +2.350000E+001로 출력
}
```

30 출력

1e 출력

0x1e 출력

0X1E 출력

23.5000 출력

2.350000E+001
출력

+2.350000E+001
출력

```
30
1e
0x1e
0X1E
23.5000
2.350000E+001
+2.350000E+001
```

포맷 멤버 함수 활용

31

```
int width(int minWidth)
```

출력되는 필드의 최소 너비를 `minWidth`로 설정하고 이전에 설정된 너비 값 리턴

```
char fill(char cFill)
```

필드의 빈칸을 `cFill` 문자로 채우도록 지정하고 이전 문자 값 리턴

```
int precision(int np)
```

출력되는 수의 유효 숫자 자리수를 `np`개로 설정. 정수 부분과 소수점 이하의 수의 자리를 모두 포함하고 소수점(.)은 제외

너비설정

```
cout.width(10); // 다음에 출력되는 "hello"를 10 칸으로 지정
cout << "Hello" << endl;
cout.width(5); // 다음에 출력되는 정수 12를 5 칸으로 지정
cout << 12 << endl;
```

1234567890

Hello

12

```
cout << '%';
cout.width(10); // 다음에 출력되는 "Korea/"만 10 칸으로 지정
cout << "Korea/" << "Seoul/" << "City" << endl;
```

1234567890

% Korea/Seoul/City

빈칸채우기

```
cout.fill('^');
cout.width(10);
cout << "Hello" << endl;
```

1234567890

^^^^Hello

유효숫자자리수

```
cout.precision(5); // 디폴트 값은 6
cout << 11./3.;
```

123456

3.6667

예제 11-6 width(), fill(), precision()을 사용한 포맷 출력

32

```
#include <iostream>
using namespace std;

void showWidth() {
    cout.width(10); // 다음에 출력되는 "hello"를 10 칸으로 지정
    cout << "Hello" << endl;
    cout.width(5); // 다음에 출력되는 정수 12를 5 칸으로 지정
    cout << 12 << endl;

    cout << '%';
    cout.width(10); // 다음에 출력되는 "Korea/"만 10 칸으로 지정
    cout << "Korea/" << "Seoul/" << "City" << endl;
}

int main() {
    showWidth();
    cout << endl;

    cout.fill('^');
    showWidth();
    cout << endl;

    cout.precision(5);
    cout << 11./3. << endl;
}
```

1234567890

Hello

12

% Korea/Seoul/City

^^^^Hello

^^^12

%^^^^Korea/Seoul/City

3.6667

조작자

33

□ 조작자

- manipulator, 스트림 조작자(stream manipulator)
- 조작자는 함수
 - C++ 표준 라이브러리에 구현된 조작자 : 입출력 포맷 지정 목적
 - 개발자 만의 조작자 작성 가능 : 다양한 목적
 - 매개 변수 **없는** 조작자와 매개 변수를 **가진** 조작자로 구분
- 조작자는 항상 << 나 >> 연산자와 함께 사용됨

□ 매개 변수 없는 조작자

```
cout << hex << showbase << 30 << endl;  
cout << dec << showpos << 100 << endl;
```

```
0x1e  
+100
```

□ 매개 변수 있는 조작자

- #include <iomanip> 필요

```
cout << setw(10) << setfill('^') << "Hello" << endl;
```

```
^^^^Hello
```

매개 변수 없는 조작자

조작자	I/O	용도
endl	0	스트림 버퍼를 모두 출력하고 다음 줄로 넘어감
oct	0	정수 필드를 8진수 기반으로 출력
dec	0	정수 필드를 10진수 기반으로 출력
hex	0	정수 필드를 16진수 기반으로 출력
left	0	왼쪽 맞춤으로 출력
right	0	오른쪽 맞춤으로 출력
fixed	0	실수 필드를 고정 소수점 방식으로 출력
scientific	0	실수 필드를 과학 산술용 방식으로 출력
flush	0	스트림 버퍼 강제 출력
showbase	0	16진수의 경우 0x로, 8진수의 경우 0을 앞에 붙여서 출력
noshowbase	0	showbase 지정 취소
showpoint	0	실수 값에 대해, 정수 부분과 소수점 이하의 끝자리 이후 남은 공간을 0으로 출력
noshowpoint	0	showpoint 지정 취소
noshowpos	0	양수인 경우 + 부호를 붙여 출력
skipws	I	입력 스트림에서 공백 문자를 읽지 않고 건너뛸
noskipws	I	skipws 지정 취소
boolalpha	0	불린 값이 출력될 때, "true" 혹은 "false" 문자열로 출력

noboolalpha

매개 변수를 가진 조작자

35

조작자	I/O	용도
<code>resetioflags(long flags)</code>	I/O	flags에 지정된 플래그들 해제
<code>setbase(int base)</code>	0	base를 출력할 수의 진수로 지정
<code>setfill(char cFill)</code>	I	필드를 출력하고 남은 공간에 cFill 문자로 채움
<code>setioflags(long flags)</code>	I/O	flags를 스트림 입출력 플래그로 설정
<code>setprecision(int np)</code>	0	출력되는 수의 유효 숫자 자리수를 np개로 설정. 소수점(.)은 별도로 카운트
<code>setw(int minWidth)</code>	0	필드의 최소 너비를 minWidth로 지정

예제 11-7 매개 변수 **없는** 조작자 사용

36

```
#include <iostream>
using namespace std;

int main() {
    cout << hex << showbase << 30 << endl;
    cout << dec << showpos << 100 << endl;
    cout << true << ' ' << false << endl;
    cout << boolalpha << true << ' ' << false << endl;
}
```

```
0x1e
+100
+1 +0
true false
```

boolalpha 조작자에 의해, "true", "false" 문자열로 출력됨

예제 11-8 매개 변수를 가진 조작자 사용 예

37

0에서 50까지 5의 배수를 10진수, 8진수, 16진수로 출력하라.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << showbase;

    // 타이틀을 출력한다.
    cout << setw(8) << "Number";
    cout << setw(10) << "Octal";
    cout << setw(10) << "Hexa" << endl;

    // 하나의 수를 십진수, 8진수, 16진수 형태로 한 줄에 출력한다.
    for(int i=0; i<50; i+=5) {
        cout << setw(8) << setfill('.') << dec << i; // 10진수
        cout << setw(10) << setfill(' ') << oct << i; // 8진수
        cout << setw(10) << setfill(' ') << hex << i << endl; // 16진수
    }
}
```

Number	Octal	Hexa
.....0	0	0
.....5	05	0x5
.....10	012	0xa
.....15	017	0xf
.....20	024	0x14
.....25	031	0x19
.....30	036	0x1e
.....35	043	0x23
.....40	050	0x28
.....45	055	0x2d

삽입 연산자(<<)

38

□ 삽입(출력) 연산자(<<)

- ▣ insertion operator, 삽입자라고도 부름
 - << 연산자는 C++의 기본 연산자 : 정수 시프트 연산자
- ▣ ostream 클래스에 삽입 연산자가 중복 작성되어 있음

```
class ostream : virtual public ios {  
    .....  
public :  
    ostream& operator<< (int n); // 정수를 출력하는 << 연산자  
    ostream& operator<< (char c); // 문자를 출력하는 << 연산자  
    ostream& operator<< (const char* s); // 문자열을 출력하는 << 연산자  
    .....  
};
```

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << 2 << 'A' << "This is a string";  
}
```

삽입 연산자의 실행 과정

cout의 스트림 버퍼

a

a123

cout << 'a' << 123;

① cout.<<('a') 호출

② cout의 operator<<(char) 함수 실행

```
ostream& operator << (char c) {
    ... 현재 스트림 버퍼에 변수 c 값 삽입
    ... 버퍼가 차면 장치에 출력
    return *this;
}
```

cout에 대한 참조 리턴

cout << 123;

③ cout.<<(123) 호출

④ cout의 operator<<(int) 함수 실행

```
ostream& operator << (int n) {
    ... 현재 스트림 버퍼에 정수 n 삽입
    ... 버퍼가 차면 장치에 출력
    return *this;
}
```

사용자 삽입 연산자 만들기

40

- 개발자가 작성한 클래스의 객체를 << 연산자로 출력

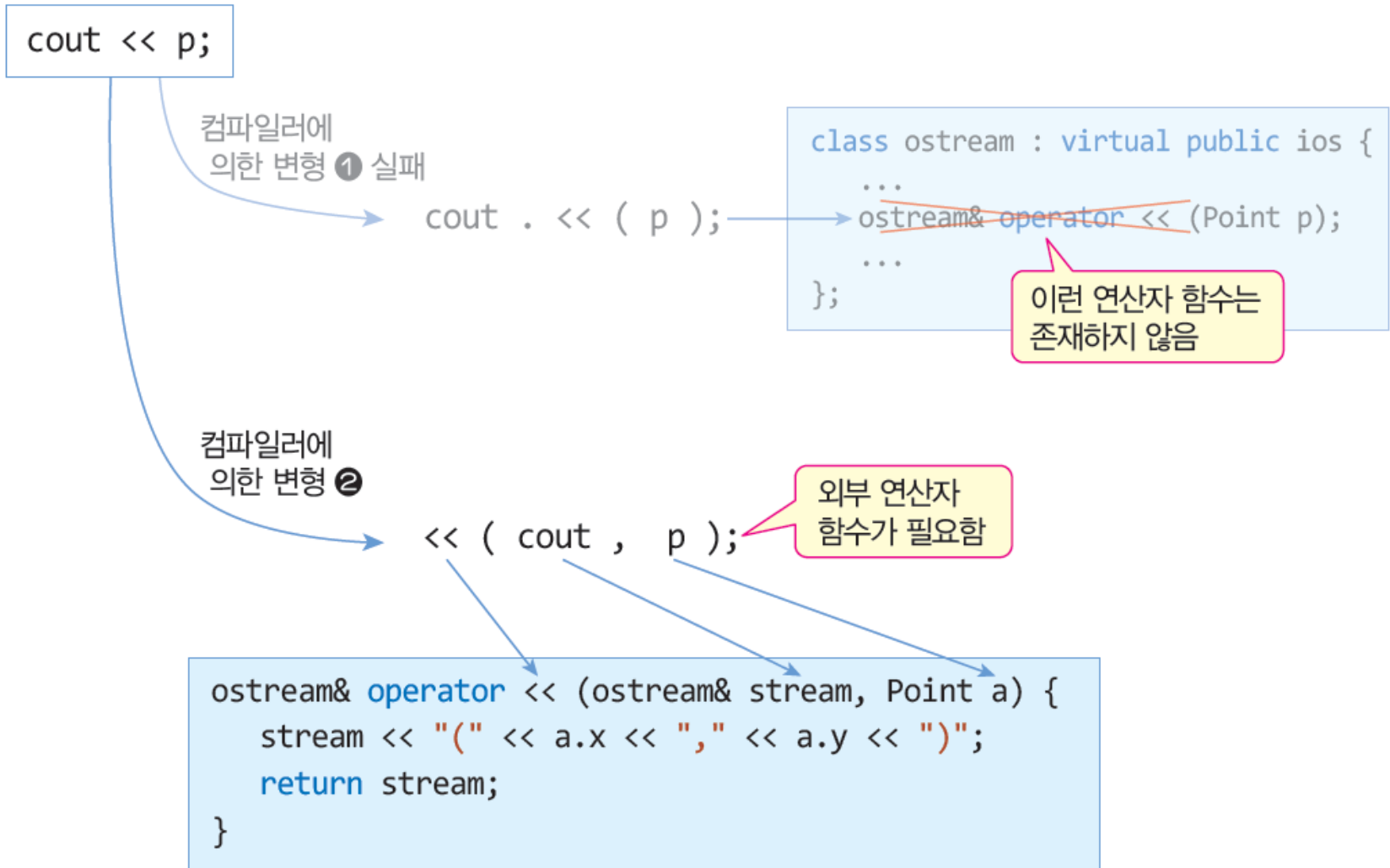
다음 Point 클래스에 대해 `cout << p;`가 가능하도록 << 연산자를 작성하라.

```
class Point {  
    int x, y;  
public:  
    Point(int x=0, int y=0) { this->x = x; this->y = y; }  
};
```

```
Point p(3,4);  
cout << p;
```

(3,4)

cout << p;를 위한 << 연산자 만들기



예제 11-9 Point 객체를 스트림에 출력하는 << 연산자 작성

42

```
#include <iostream>
using namespace std;

class Point { // 한 점을 표현하는 클래스
    int x, y; // private 멤버
public:
    Point(int x=0, int y=0) {
        this->x = x;
        this->y = y;
    }
    friend ostream& operator << (ostream& stream, const Point& a);
};

// << 연산자 함수
ostream& operator << (ostream& stream, const Point& a) {
    stream << "(" << a.x << "," << a.y << ")";
    return stream;
}

int main() {
    Point p(3,4); // Point 객체 생성
    cout << p << endl; // Point 객체 화면 출력

    Point q(1,100), r(2,200); // Point 객체 생성
    cout << q << r << endl; // Point 객체들 연속하여 화면 출력
}
```

private 필드 x, y를 접근하기 위해
이 함수를 Point 클래스에
friend로 선언함.

(3,4)
(1,100)(2,200)

예제 11-10 Book 클래스를 만들고 Book 객체를 스트림에 출력하는 << 연산자 작성

```
#include <iostream>
#include <string>
using namespace std;

class Book { // 책을 표현하는 클래스
    string title;
    string press;
    string author;
public:
    Book(string title="", string press="", string author="") {
        this->title = title;
        this->press = press;
        this->author = author;
    }
    friend ostream& operator << (ostream& stream, const Book& b); // friend 선언
};

// << 연산자 함수
ostream& operator << (ostream& stream, const Book& b) {
    stream << b.title << "," << b.press << "," << b.author;
    return stream;
}

int main() {
    Book book("소유냐 존재냐", "한국출판사", "에리히프롬"); // Book 객체 생성
    cout << book; // Book 객체 book 화면 출력
}
```

소유냐 존재냐, 한국출판사, 에리히프롬

추출 연산자(>>)

44

□ 추출(입력) 연산자(>>)

□ extraction operator

- >> 연산자는 C++의 기본 연산자 : 정수 시프트 연산자

□ ostream 클래스에 추출 연산자가 중복 작성되어 있음

```
class istream : virtual public ios {
    .....
public :
    istream& operator>> (int& n); // 정수를 입력하는 >> 연산자
    istream& operator>> (char& c); // 문자를 입력하는 >> 연산자
    istream& operator>> (const char* s); // 문자열을 입력하는 >> 연산자
    .....
};
```

□ 추출 연산자의 실행 과정

- 삽입 연산자의 실행 과정과 유사하므로 생략

```
int main() {
    int i; char c; char buf[128];
    cin >> i >> c >> buf;
}
```

사용자 추출 연산자 만들기

45

- 개발자가 작성한 클래스의 객체에 >> 연산자로 입력

다음 Point 클래스에 대해 cin >> p;가 가능하도록 >> 연산자를 작성하라.

```
class Point {  
    int x, y;  
public:  
    Point(int x=0, int y=0) { this->x = x; this->y = y; }  
};
```

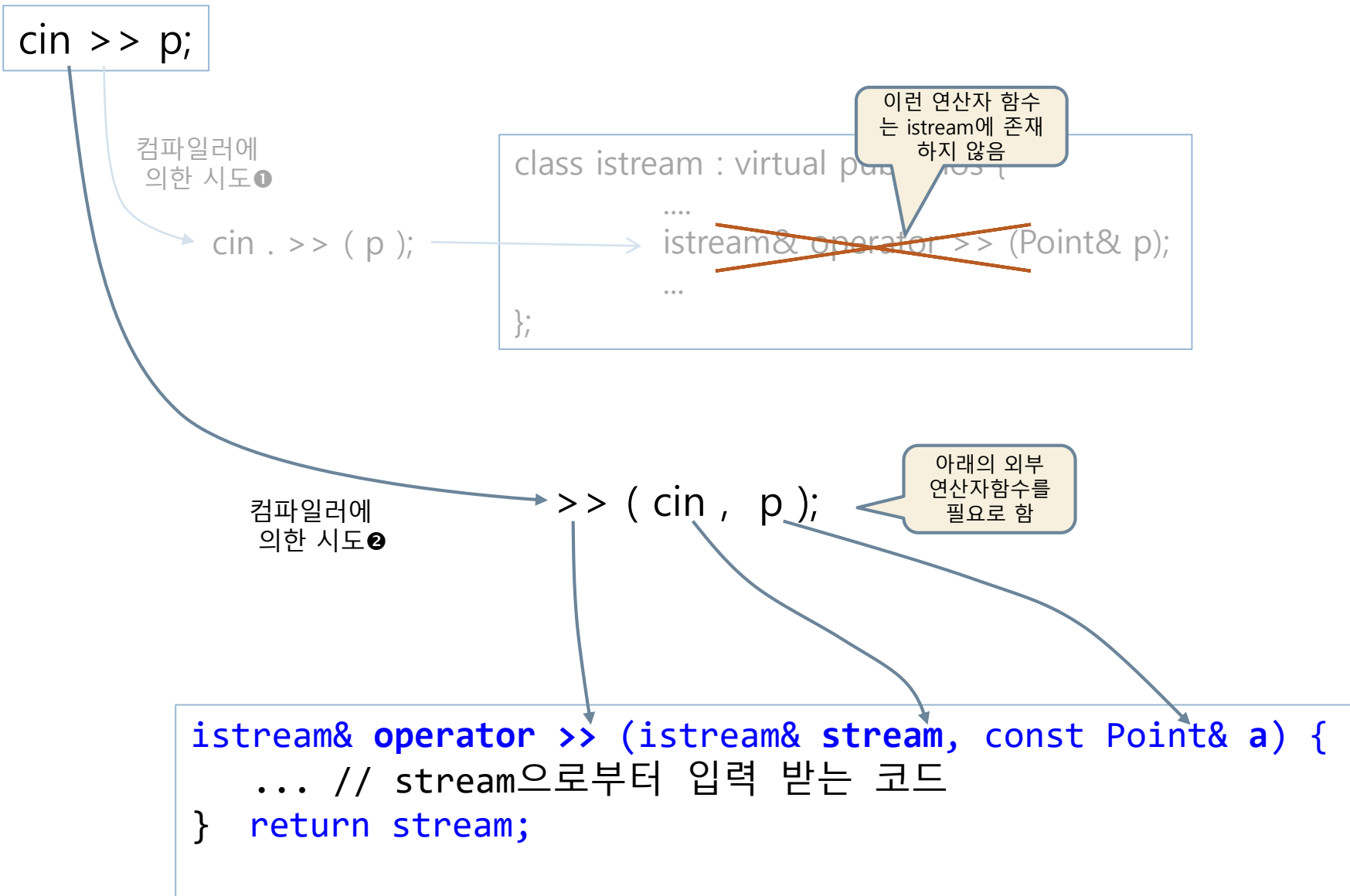
```
Point p;  
cin >> p;  
cout << p;
```

```
x 좌표>>100  
y 좌표>>200  
(100,200)
```

cin >> p 실행

cout << p 실행

cin >> p;를 위한 >> 연산자 만들기



예제 11-11 Point 객체를 입력 받는 >> 연산자 작성

```
#include <iostream>
using namespace std;

class Point { // 한 점을 표현하는 클래스
    int x, y; // private 멤버
public:
    Point(int x=0, int y=0) {
        this->x = x;
        this->y = y;
    }
    friend istream& operator >> (istream& ins, Point &a); // friend 선언
    friend ostream& operator << (ostream& stream, const Point& a); // friend 선언
};
```

```
istream& operator >> (istream& ins, Point &a) { // >> 연산자 함수
    cout << "x 좌표>>";
    ins >> a.x;
    cout << "y 좌표>>";
    ins >> a.y;
    return ins;
}
```

```
ostream& operator << (ostream& stream, const Point& a) { // << 연산자 함수
    stream << "(" << a.x << "," << a.y << ")";
    return stream;
}
```

```
int main() {
    Point p; // Point 객체 생성
    cin >> p; // >> 연산자 호출하여 x 좌표와 y 좌표를 키보드로 읽어 객체 p 완성
    cout << p; // << 연산자 호출하여 객체 p 출력
}
```

x 좌표>>100
y 좌표>>200
(100,200)

cin >> p 실행

cout << p 실행

조작자 실행 과정

```
cout << endl;
```

컴파일러에 의해 변형

```
cout . << ( endl );
```

호출

endl 함수 주소 전달

```
class ostream : virtual public ios {  
public:  
    ostream& operator << (ostream& (* _f)(ostream&));  
    ...  
};
```

```
ostream& ostream::operator << (ostream& (* _f)(ostream&)) {  
    (*_f)(*this); // *this는 cout  
    return *this; // cout의 참조 리턴  
}
```

endl(cout) 호출

cout 전달

```
ostream& endl(ostream& outs) {  
    outs.put('\n'); // 개행 문자 삽입  
    outs.flush(); // 버퍼 강제 출력  
    return outs; // 출력 스트림의 참조 리턴  
}
```


사용자 정의 조작자 함수 원형

49

□ 매개 변수 없는 조작자의 경우

```
istream& manipulatorFunction (istream& ins)
```

입력 스트림에 사용되는 조작자 원형

```
ostream& manipulatorFunction (ostream& outs)
```

출력 스트림에 사용되는 조작자 원형

예제 11-12 출력용 사용자 정의 조작자 만들기

50

```
#include <iostream>
using namespace std;

ostream& fivestar(ostream& outs) {
    outs << "*****";
    return outs;
}

ostream& rightarrow(ostream& outs) {
    return outs << "---->";
}

ostream& beep(ostream& outs) {
    return outs << '\a'; // 시스템 beep(뽕 소리) 발생
}

int main() {
    cout << "벨이 울립니다" << beep << endl;
    cout << "C" << rightarrow << "C++" << rightarrow << "Java" << endl;
    cout << "Visual" << fivestar << "C++" << endl;
}
```

벨이 울립니다
C---->C++---->Java
Visual*****C++

뽕 소리 남

예제 11-13 입력용 사용자 정의 조작자 만들기

51

```
#include <iostream>
#include <string>
using namespace std;
```

조작자 작성

```
istream& question(istream& ins) {
    cout << "거울아 거울아 누가 제일 예쁘니?";
    return ins;
}
```

조작자 사용

```
int main() {
    string answer;
    cin >> question >> answer;
    cout << "세상에서 제일 예쁜 사람은 " << answer << "입니다." << endl;
}
```

거울아 거울아 누가 제일 예쁘니?백설공주
세상에서 제일 예쁜 사람은 백설공주입니다.